

嵌入式電腦視覺系統

W9 講義：CameraX 即時串流 + 用 W6/W7 古典 CV 完成猜拳辨識

謝昇憲 George Hsieh

學習路線大綱

第 1 章：W9 課程定位與本週限制

↓

第 2 章：從第 13 章 ImageAnalysis 接續

↓

第 3 章：手部 ROI 抽取（HSV 膚色 + 形態學）

↓

第 4 章：輪廓抽取與形狀分析（W6 工具整合）

↓

第 5 章：古典 RPS 判斷邏輯（凸缺陷 + 指縫數法）

↓

第 6 章：CameraX 即時 RPS 完整整合

↓

第 7 章：替代方案 — 用 W7 ORB 樣板比對（選做）

↓

第 8 章：本週作業與常見錯誤排除

第 1 章 W9 課程定位與本週限制

1.1 本週在整學期的位置

本學期前半段我們已經走完古典 CV 三部曲：

W6：像素 → 結構（邊緣、Hough、形態學、輪廓特徵）

W7：結構 → 指紋（Harris/FAST、SIFT、ORB、matching、RANSAC）

W8：期末專題方向簡報（學生主導）

W9 開始進入「同樣的問題（猜拳辨識），用不同方法跑一次」的對照系列：

W9：CameraX + W6/W7 古典工具（本週）

W10：CameraX + ML / DL 訓練模型

W11：CameraX + YOLO + Tracking 整合

W12：CameraX + MediaPipe 骨架

W13：補課 / 進階主題

W14：CV 新研究與閱讀地圖

1.2 本週限制：只能用 W6 / W7 學過的工具

為了讓對照有意義，本週實作有以下限制：

可以用：W6 教過的邊緣偵測、形態學、輪廓、形狀特徵；W7 教過的 Harris/FAST、SIFT、ORB、ratio test、RANSAC

可以用：HSV 色彩轉換、threshold、cv2.findContours、cv2.convexHull、cv2.convexityDefects

不可以用：CNN (含 MobileNet、ResNet)、YOLO、MediaPipe、OpenPose、任何 deep learning model

不可以用：sklearn 任何含「訓練資料」的方法 (這些屬於 W10 範圍)

本週的目的不是做出最高正確率，而是讓你親手體驗「古典 CV 在語意辨識任務上的天花板」。預期正確率落在 60% - 80% 之間，W10 ML/DL 會把這個數字往上拉。

1.3 本週實作目標

完成以下系統：

Android 手機相機

↓ CameraX ImageAnalysis (即時串流)

ImageProxy (YUV_420_888)

↓ Java : imageProxyToNv21()

NV21 byte[]

↓ Chaquopy 呼叫 Python

Python : rps_classical.process_frame()

↓

HSV 膚色偵測 → 形態學清雜訊 → 最大輪廓 → 凸包 → 凸缺陷

↓

指縫數 → 分類為「石頭 / 剪刀 / 布」

↓ 回傳 (label, score) 給 Java

ImageView / TextView 顯示結果

1.4 為什麼從古典開始？

如果一開始就讓學生用 MobileNetV2 跑 RPS，會得到 90%+ 的正確率，但學生不會理解：

為什麼這個任務看起來「簡單」，古典方法卻吃力？

什麼情況下手工特徵會失效？光線變化？背景複雜？多手出現？

AI 模型實際上學到了什麼，而古典方法做不到？

先用古典走一次，你就會自然回答這些問題。下週進入 W10 ML/DL 時，每一個準確率提升你都看得得到「來自哪裡」。

第 2 章 從第 13 章 ImageAnalysis 接續

2.1 W8 講義已經給的基礎

第 8 週的 Android Chaquopy + OpenCV + CameraX 講義，已經完成以下章節：

第 0-9 章：Chaquopy 基礎，Android 與 Python 資料傳遞，用靜態圖片做 OpenCV 處理

第 10 章：CameraX 基礎 (PreviewView + ImageCapture 單張擷取)

第 11 章：ImageProxy 與 YUV_420_888 → NV21 格式轉換

第 12 章：CameraX + Chaquopy + OpenCV 整合 (以 Canny 為例)

第 13 章：進階 — ImageAnalysis 即時影像串流

W9 完全建立在這些基礎上。在開始本週實作前，請先確認：

能成功用 CameraX Preview 看到相機畫面
能呼叫 Python 函式並接到回傳的 byte[]
理解 NV21 與 BGR 的轉換
ImageAnalysis 的 analyzer 流程可以接到 Python 端

2.2 W9 的起點：第 13 章的「空殼」

回顧第 13 章的 ImageAnalysis 結構：

```
imageAnalysis = new ImageAnalysis.Builder()
    .setTargetResolution(new Size(640, 480))
    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
    .build();
```

```
imageAnalysis.setAnalyzer(cameraExecutor, image -> {
    frameCount++;
    if (frameCount % 5 != 0) {
        image.close();
        return;
    }
}
```

```
// === 在此處呼叫 Python 處理 ===
// 第 13 章只用 Canny 做示範
// W9 要把這裡換成「真正的偵測 pipeline」
```

```
image.close();
});
```

```
cameraProvider.bindToLifecycle(this, selector, preview, imageAnalysis);
```

W9 的核心工作就是「填滿那一段註解」——把 Canny 換成完整的 RPS pipeline。

2.3 W9 整體 Pipeline 預覽

本週的處理流程一張圖看完：

```
ImageAnalysis analyzer
↓
ImageProxy ( YUV_420_888 )
↓
Java imageProxyToNv21(image)
↓
PyObject result = py.getModule("rps_classical")
    .callAttr("process_frame", nv21, w, h)
↓
Python rps_classical.process_frame(nv21, w, h):
├─ 1. NV21 → BGR
├─ 2. BGR → HSV
├─ 3. HSV 膚色 mask
└─ 4. 形態學 open + close
```

- └ 5. findContours → 取最大輪廓
- └ 6. convexHull + convexityDefects
- └ 7. 計算指縫數
- └ 8. 分類為 rock / scissors / paper

↓

回傳 (label, n_fingers, confidence) 給 Java

↓

runOnUiThread → TextView 更新「目前判斷：剪刀」

Python 端集中在 rps_classical.py 這一個檔案，Java 端只負責 frame 抓取與顯示。

第 3 章 手部 ROI 抽取 (Pipeline 第一步)

3.1 為什麼要先分割出「手」？

整張 1080p 影像中，手只佔大約 20% - 40% 面積，其他都是背景、衣服、桌面、其他物件。

如果直接對整張影像做輪廓 / 凸包分析，結果會被背景物件干擾。「先把手找出來」是所有古典手勢辨識的第一步。

3.2 三種手部偵測方案比較

在開始實作前，先理解三種主流方案的取捨：

3.2.1 方案 A：純 threshold (最簡單)

做法：灰階 + Otsu threshold，假設手是「亮的、背景是暗的」

優點：3 行程式碼解決

缺點：對光線、膚色、背景顏色極度敏感，幾乎只能在「黑布襯底 + 強光」下用

適用：教學示範第一版 (dummy baseline)

3.2.2 方案 B：HSV 膚色偵測 + 形態學 (本週採用)

做法：BGR → HSV，用「色相 H 在膚色範圍 + 飽和度 S 與明度 V 適中」做 mask

優點：對光線變化適度容忍，跨膚色也能調整

缺點：手臂、臉也會被誤抓；木質桌面 (接近膚色) 容易誤判

適用：本週主菜

3.2.3 方案 C：背景減法 (Background Subtraction)

做法：先抓 100 frames 不含手的背景模型，新 frame 減背景 → 動的東西就是前景

優點：與膚色無關，跨膚色完全相容

缺點：需要「先讓鏡頭看到沒手的場景」初始化；相機晃動就要重來

適用：固定鏡頭場景 (譬如手放在固定位置)

本週採方案 B。如果你想在期末專題挑戰方案 C，OpenCV 的 `cv2.createBackgroundSubtractorMOG2()` 是現成工具。

3.3 HSV 膚色範圍的選擇

膚色在 HSV 空間集中於：

H (色相)：0 - 20 度 (偏紅 / 黃)

S (飽和度) : 30 - 150 (不能太低 = 灰色 , 也不能太高 = 鮮豔)

V (明度) : 60 - 255 (不能太暗)

OpenCV 的 H 範圍是 0 - 179 (不是 0 - 360) , 所以實際參數通常設 :

```
lower_skin = np.array([0, 30, 60], dtype=np.uint8)
```

```
upper_skin = np.array([20, 150, 255], dtype=np.uint8)
```

```
mask = cv2.inRange(hsv, lower_skin, upper_skin)
```

不同膚色 / 不同光源下 , 這組參數要微調。教學時建議讓學生「手動調出最佳值」當作練習。

3.4 形態學清理 mask

cv2.inRange 出來的 mask 通常有兩種問題 :

外圍有許多小亮點雜訊 (背景中有些像素剛好符合膚色範圍)

手內部有零散黑點 (光線不均造成局部脫離膚色範圍)

用 W6 教過的形態學處理 :

```
# 去除外圍小雜訊 : open ( 先 erode 後 dilate )
```

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=2)
```

```
# 填補手內小洞 : close ( 先 dilate 後 erode )
```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
```

```
# 額外稍微擴張 , 確保指縫不被吃掉
```

```
mask = cv2.dilate(mask, kernel, iterations=1)
```

3.5 Python 完整實作 : detect_hand_roi()

```
# rps_classical.py ( 第一段 )
```

```
import cv2
```

```
import numpy as np
```

```
# 膚色範圍 ( 學生可依教室光線微調 )
```

```
LOWER_SKIN = np.array([0, 30, 60], dtype=np.uint8)
```

```
UPPER_SKIN = np.array([20, 150, 255], dtype=np.uint8)
```

```
KERNEL = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

```
def detect_hand_mask(bgr):
```

```
    """
```

```
    輸入 : BGR 影像
```

```
    輸出 : 二值化 mask ( uint8 , 0/255 ) , 其中 255 表示膚色 ( 手 )
```

```
    """
```

```
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
```

```
    mask = cv2.inRange(hsv, LOWER_SKIN, UPPER_SKIN)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, KERNEL, iterations=2)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, KERNEL, iterations=2)
```

```
mask = cv2.dilate(mask, KERNEL, iterations=1)
return mask
```

第 4 章 輪廓抽取與形狀分析 (W6 工具整合)

4.1 從 mask 到「手的輪廓」

mask 是二值圖，下一步用 cv2.findContours 把所有連通區的邊界抽出來。

問題：mask 中可能有手 + 手臂 + 偶爾誤偵測的小區塊，但我們只關心「最大那一個」（通常是手 + 手臂）。

```
def find_largest_contour(mask):
    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not contours:
        return None
    # 用面積排序找最大
    largest = max(contours, key=cv2.contourArea)
    if cv2.contourArea(largest) < 5000:
        return None # 太小，可能沒手
    return largest
```

4.2 凸包 (Convex Hull) 的計算

凸包是「能包住所有輪廓點的最小凸多邊形」。

直覺：用橡皮筋圈住手 → 橡皮筋輪廓就是凸包。

凸包本身對 RPS 分類意義不大，重點是「凸包 vs 真實輪廓的差」——也就是凸缺陷 (convexity defects)。

```
hull_indices = cv2.convexHull(contour, returnPoints=False)
hull_points = cv2.convexHull(contour, returnPoints=True)
defects = cv2.convexityDefects(contour, hull_indices)
```

注意參數差異：算 defects 時 returnPoints 要設 False (回傳的是 index 而不是座標)。

4.3 凸缺陷 (Convexity Defects) 是什麼？

凸缺陷是「凸包邊」與「真實輪廓」之間的內凹區域。每個 defect 由四個值表示：

start_index：凸包邊起點在 contour 中的 index
end_index：凸包邊終點在 contour 中的 index
far_index：凸缺陷中「最深的那一點」的 index
depth：缺陷深度 (從凸包邊到 far 點的距離 × 256)

視覺上，每兩個手指之間就是一個凸缺陷 (因為手指外凸、指縫內凹)。

4.4 為什麼凸缺陷可以數指縫？

正常打開的手有 5 根手指、4 個指縫。每個指縫就是一個有意義的凸缺陷。

石頭：拳頭，輪廓接近圓形，凸缺陷淺且少（通常 0 個有意義的）

剪刀：2 指伸出，中間有 1 個明顯指縫（1 個有意義的凸缺陷）

布：5 指張開，有 4 個指縫（4 個有意義的凸缺陷）

「有意義的凸缺陷」要過濾掉手腕、雜訊造成的小缺陷。靠兩個條件：

深度足夠大（譬如 $\text{depth}/256 > 30$ 像素）

角度小於 90 度（指縫兩側手指夾角通常 $< 90^\circ$ ）

4.5 完整實作：count_fingers()

```
def count_fingers(contour, defects):
    """
    輸入：手輪廓 + cv2.convexityDefects 回傳的 defects
    輸出：判斷出的指縫數量（int）
    """
    if defects is None:
        return 0

    finger_gaps = 0
    for i in range(defects.shape[0]):
        s, e, f, d = defects[i, 0]
        start = tuple(contour[s][0])
        end = tuple(contour[e][0])
        far = tuple(contour[f][0])
        depth = d / 256.0

        # 條件 1：深度夠大才算
        if depth < 30:
            continue

        # 條件 2：用餘弦定理算 far 點兩側的夾角
        a = np.linalg.norm(np.array(start) - np.array(end))
        b = np.linalg.norm(np.array(start) - np.array(far))
        c = np.linalg.norm(np.array(end) - np.array(far))
        angle = np.degrees(np.arccos((b**2 + c**2 - a**2) / (2 * b * c + 1e-6)))

        if angle < 90:
            finger_gaps += 1

    return finger_gaps
```

第 5 章 古典 RPS 判斷邏輯（指縫數法）

5.1 分類規則：簡單但有效

得到指縫數後，用三個閾值決定 RPS 類別：

```
def classify_rps(n_gaps):
    if n_gaps <= 1:
```



```

    return "rock"      # 石頭：0 個指縫
elif n_gaps == 2:
    return "scissors"  # 剪刀：通常剛好 1 個指縫，但角度寬時可達 2
else:
    return "paper"     # 布：3-4 個指縫

```

這個簡單規則的預期表現：

石頭：辨識最穩定，通常 80-90% 正確

剪刀：中等，70-80%（指縫角度容易誤判）

布：中等，65-80%（手腕誤判、指縫被吃掉等問題）

整體：通常 70-80%（需要光線、背景配合）

5.2 視覺化偵測中間結果（debug 用）

開發階段強烈建議在影像上把以下東西畫出來：

手輪廓（綠色）

凸包（紅色）

凸缺陷的 far 點（藍色實心圓）

每個有意義缺陷的角度數值（疊在 far 點旁）

這樣學生可以「看見」演算法在想什麼，自己調參數時有依據。

```

def draw_debug(bgr, contour, defects, n_gaps):
    out = bgr.copy()
    cv2.drawContours(out, [contour], -1, (0, 255, 0), 2)

    hull = cv2.convexHull(contour, returnPoints=True)
    cv2.drawContours(out, [hull], -1, (0, 0, 255), 2)

    if defects is not None:
        for i in range(defects.shape[0]):
            s, e, f, d = defects[i, 0]
            far = tuple(contour[f][0])
            depth = d / 256.0
            if depth > 30:
                cv2.circle(out, far, 6, (255, 0, 0), -1)

    label = classify_rps(n_gaps)
    cv2.putText(out, f"{{label}} (gaps={{n_gaps}})",
                (20, 40), cv2.FONT_HERSHEY_SIMPLEX,
                1.0, (0, 255, 255), 2)
    return out

```

5.3 主流程：process_frame()

把前面所有步驟串起來，就是 W9 的核心 Python 函式：

```

def process_frame(nv21_bytes, w, h):
    """

```



```

Chaquopy 從 Java 呼叫的入口。
輸入：NV21 byte array + 寬高
輸出：(label, n_gaps, jpeg_bytes)
    label：rock/scissors/paper/none 字串
    n_gaps：偵測到的指縫數
    jpeg_bytes：debug 視覺化的 JPEG byte array
"""
# NV21 → BGR
yuv = np.frombuffer(bytes(nv21_bytes), dtype=np.uint8) \
    .reshape((h * 3 // 2, w))
bgr = cv2.cvtColor(yuv, cv2.COLOR_YUV2BGR_NV21)

# 1. 膚色偵測
mask = detect_hand_mask(bgr)

# 2. 最大輪廓
contour = find_largest_contour(mask)
if contour is None:
    # 沒抓到手 → 直接回傳原圖
    ok, buf = cv2.imencode(".jpg", bgr)
    return ("none", 0, buf.tobytes())

# 3. 凸包 + 凸缺陷
hull_idx = cv2.convexHull(contour, returnPoints=False)
if hull_idx is None or len(hull_idx) < 3:
    ok, buf = cv2.imencode(".jpg", bgr)
    return ("none", 0, buf.tobytes())

defects = cv2.convexityDefects(contour, hull_idx)

# 4. 指縫數
n_gaps = count_fingers(contour, defects)

# 5. 分類
label = classify_rps(n_gaps)

# 6. 視覺化
debug_img = draw_debug(bgr, contour, defects, n_gaps)

# 7. 編碼回 JPEG
ok, buf = cv2.imencode(".jpg", debug_img,
    [cv2.IMWRITE_JPEG_QUALITY, 70])
return (label, n_gaps, buf.tobytes())

```

第 6 章 CameraX 即時 RPS 完整整合

6.1 完整 Python 模組：rps_classical.py

把第 3、4、5 章的程式整合在同一個檔案，放置位置：

app/src/main/python/rps_classical.py

這個模組對外只暴露一個函式 process_frame()，其他都是內部輔助。

6.2 Java 端：MainActivity 整合 ImageAnalysis

完整 onCreate 與 analyzer 邏輯（沿用 W8 第 13 章 ImageAnalysis 結構）：

```
package com.example.rpsclassical;

import android.Manifest;
import android.content.pm.PackageManager;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Bundle;
import android.util.Log;
import android.util.Size;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.camera.core.CameraSelector;
import androidx.camera.core.ImageAnalysis;
import androidx.camera.core.ImageProxy;
import androidx.camera.core.Preview;
import androidx.camera.lifecycle.ProcessCameraProvider;
import androidx.camera.view.PreviewView;
import androidx.core.content.ContextCompat;

import com.chaquo.python.PyObject;
import com.chaquo.python.Python;
import com.chaquo.python.android.AndroidPlatform;
import com.google.common.util.concurrent.ListenableFuture;

import java.nio.ByteBuffer;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MainActivity extends AppCompatActivity {

    private PreviewView previewView;
    private ImageView resultView;
    private TextView labelView;
    private ExecutorService cameraExecutor;
    private Python py;
    private int frameCount = 0;
```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    previewView = findViewById(R.id.previewView);
    resultView = findViewById(R.id.resultView);
    labelView = findViewById(R.id.labelView);

    if (!Python.isStarted()) {
        Python.start(new AndroidPlatform(this));
    }
    py = Python.getInstance();

    cameraExecutor = Executors.newSingleThreadExecutor();

    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
        == PackageManager.PERMISSION_GRANTED) {
        startCamera();
    } else {
        cameraPermLauncher.launch(Manifest.permission.CAMERA);
    }
}

private final ActivityResultLauncher<String> cameraPermLauncher =
    registerForActivityResult(new ActivityResultContracts.RequestPermission(),
        isGranted -> {
            if (isGranted) startCamera();
            else Toast.makeText(this, "需要攝影機權限", Toast.LENGTH_LONG).show();
        });

private void startCamera() {
    ListenableFuture<ProcessCameraProvider> future =
        ProcessCameraProvider.getInstance(this);

    future.addListener(() -> {
        try {
            ProcessCameraProvider cameraProvider = future.get();

            Preview preview = new Preview.Builder().build();
            preview.setSurfaceProvider(previewView.getSurfaceProvider());

            ImageAnalysis imageAnalysis = new ImageAnalysis.Builder()
                .setTargetResolution(new Size(640, 480))
                .setBackpressureStrategy(
                    ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                .build();

            imageAnalysis.setAnalyzer(cameraExecutor, this::analyze);
        }
    });
}

```

```

        CameraSelector selector = CameraSelector.DEFAULT_BACK_CAMERA;
        cameraProvider.unbindAll();
        cameraProvider.bindToLifecycle(
            this, selector, preview, imageAnalysis);

    } catch (Exception e) {
        Log.e("CameraX", "Failed to bind camera", e);
    }
}, ContextCompat.getMainExecutor(this));
}

private void analyze(ImageProxy image) {
    frameCount++;
    // 每 5 個 frame 處理一次 (節省效能)
    if (frameCount % 5 != 0) {
        image.close();
        return;
    }

    int w = image.getWidth();
    int h = image.getHeight();
    byte[] nv21 = imageProxyToNv21(image);
    image.close();

    try {
        PyObject module = py.getModule("rps_classical");
        PyObject result = module.callAttr("process_frame", nv21, w, h);

        // result = (label, n_gaps, jpeg_bytes)
        String label = result.asList().get(0).toString();
        int gaps = result.asList().get(1).toInt();
        byte[] jpeg = result.asList().get(2).toJava(byte[].class);

        Bitmap bmp = BitmapFactory.decodeByteArray(jpeg, 0, jpeg.length);

        runOnUiThread(() -> {
            resultView.setImageBitmap(bmp);
            labelView.setText(String.format(
                "目前判斷： %s ( 指縫=%d ) ", label, gaps));
        });

    } catch (Exception e) {
        Log.e("RPS", "Python call failed", e);
    }
}

// imageProxyToNv21() 沿用 W8 第 11 章的實作
// ... 略 ...

```

```

@Override
protected void onDestroy() {
    super.onDestroy();
    if (cameraExecutor != null) cameraExecutor.shutdown();
}
}

```

6.3 activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.camera.view.PreviewView
        android:id="@+id/previewView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toTopOf="@id/labelView" />

    <ImageView
        android:id="@+id/resultView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:adjustViewBounds="true"
        android:scaleType="fitCenter"
        app:layout_constraintTop_toTopOf="@id/previewView"
        app:layout_constraintBottom_toBottomOf="@id/previewView"
        app:layout_constraintStart_toStartOf="@id/previewView"
        app:layout_constraintEnd_toEndOf="@id/previewView" />

    <TextView
        android:id="@+id/labelView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="目前判斷：—"
        android:textSize="22sp"
        android:padding="16dp"
        android:gravity="center"
        app:layout_constraintBottom_toBottomOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

6.4 app/build.gradle 設定

Chaquopy 端需要安裝 numpy 與 opencv-python：

```
chaquopy {
    defaultConfig {
        version "3.10"
        pip {
            install "numpy"
            install "opencv-python"
        }
    }
}
```

CameraX 依賴沿用 W8 第 10 章 (camerax-core / camera2 / lifecycle / view 各一個)。

6.5 FPS 與效能調整

CameraX ImageAnalysis 預設可以跑到 30 FPS，但 Python 端跑完整 RPS pipeline 通常只能達 5 - 10 FPS (在 Pi 4 / 中低階手機上)。

調整建議：

降低解析度：640×480 → 320×240 可以再快 3 倍

frameCount % 5 == 0 控制呼叫頻率 (本講義已用)

STRATEGY_KEEP_ONLY_LATEST 確保不堆積 frame

Python 端不要每 frame encode PNG (用 JPEG quality=70 比較快)

debug 視覺化可以在 release 版關掉，純回傳 label

第 7 章 替代方案 — 用 W7 ORB 樣板比對 (選做)

7.1 為什麼提這個替代方案？

指縫數法的優點是「不需要訓練資料」，但弱點是「對手勢角度敏感」(同樣的剪刀，斜著比、正著比可能算出不同的指縫數)。

W7 學過的 ORB 提供另一條思路：把三個樣板 (rock.jpg、scissors.jpg、paper.jpg) 的 ORB descriptors 預先抽出，每個 frame 與三個樣板比對 → 最像的就是答案。

7.2 樣板準備

資料夾內已經有 rock.jpg、scissors.jpg、paper.jpg 三張典型手勢樣板。

把它們複製到 Android 的 Python 資源目錄：

```
app/src/main/python/templates/rock.jpg
app/src/main/python/templates/scissors.jpg
app/src/main/python/templates/paper.jpg
```

7.3 ORB 比對程式

```
# rps_orb.py
import os
import cv2
import numpy as np

ORB = cv2.ORB_create(nfeatures=500)
BF = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)

# 預先抽樣板特徵 ( 模組載入時跑一次 )
BASE = os.path.dirname(os.path.abspath(__file__))
TEMPLATES = {}
for label in ["rock", "scissors", "paper"]:
    img = cv2.imread(os.path.join(BASE, "templates", f"{label}.jpg"),
                     cv2.IMREAD_GRAYSCALE)
    kp, des = ORB.detectAndCompute(img, None)
    TEMPLATES[label] = (kp, des)

def classify_by_orb(bgr):
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    kp_q, des_q = ORB.detectAndCompute(gray, None)
    if des_q is None or len(des_q) < 10:
        return ("none", 0)

    best_label, best_count = "none", 0
    for label, (kp_t, des_t) in TEMPLATES.items():
        matches = BF.knnMatch(des_q, des_t, k=2)
        # Lowe's ratio test
        good = [m for m, n in matches if m.distance < 0.75 * n.distance]
        if len(good) > best_count:
            best_label = label
            best_count = len(good)

    return (best_label, best_count)
```

7.4 兩種方法的比較

指縫數法：對「手張開角度」敏感、對「樣板差異」不敏感；計算便宜

ORB 樣板比對：對「整體形狀」敏感、對「光線/旋轉」較容忍；計算貴 2-3 倍
實務上可以兩個都跑、投票決定 → 正確率可上升到 80-85%

7.5 進階：RANSAC 過濾不可靠的 match

W7 學過的 RANSAC + Homography 也可以用在這裡，過濾「描述子相似但幾何位置不合理」的錯配。本講義不詳細展開，留作期末專題挑戰。

第 8 章 本週作業與常見錯誤排除

8.1 本週作業 (HW-W9)

完成一個 Android App，達成以下基本要求：

使用 CameraX ImageAnalysis 即時抓取相機畫面

用 Chaquopy 呼叫 Python `rps_classical.process_frame()`

在畫面上即時顯示判斷結果 (rock / scissors / paper)

在固定背景下，三種手勢的正確率達 60% 以上

加分項 (任選)：

(加分 1) 整合 ORB 樣板比對作為「第二意見」，兩種方法投票

(加分 2) 加入「畫面凍結 0.5 秒後才更新結果」減少抖動

(加分 3) 整合 W6 學過的 `cv2.findContours` 屬性 (面積、緊密度) 作為輔助特徵

(加分 4) UI 上顯示 debug 視覺化 (輪廓、凸包、指縫圓點)

8.2 繳交內容

完整 Android Studio 專案 (.zip)

一段 30 秒的測試影片 (自拍三種手勢，標記正確率)

一份 README.md：說明你做了哪些加分項、遇到什麼問題

8.3 常見錯誤排除

8.3.1 膚色 mask 抓不到手

檢查光線：教室太暗時 V 值不夠

微調 `LOWER_SKIN / UPPER_SKIN` 的 H 範圍 (不同膚色不一樣)

用 `cv2.imshow("mask", mask)` 在桌機端先驗證閾值

8.3.2 找到的輪廓有手 + 手臂

如果只關心手部，可以加一個「面積上限」過濾掉太大的輪廓，或要求手只放在畫面上半部。期末專題可以挑戰「自動分離手與手臂」。

8.3.3 凸缺陷一直是 None

原因通常是 `cv2.convexHull` 用 `returnPoints=True` 算出來的 hull 點傳給 `convexityDefects`，這是錯的。要用 `returnPoints=False` 拿 `indices`。

8.3.4 FPS 太慢

降低解析度到 320×240

`frameCount` 控制成 `% 10 == 0`

檢查 LogCat 中是否有 Python warning (特別是 GIL 釋放警告)

8.3.5 結果抖動劇烈

每 5 個 frame 處理一次本身會造成判斷跳動。可以加一個「最近 5 次判斷的多數決」— 簡單的 buffer 平滑：

```
from collections import deque, Counter
```

```
label_history = deque(maxlen=5)
```

```
# 在 process_frame 結尾：
```

```
label_history.append(label)
```

```
smoothed = Counter(label_history).most_common(1)[0][0]
```

```
return (smoothed, n_gaps, buf.tobytes())
```

8.4 下週預告 (W10)

下週同樣的題目 (CameraX + RPS) , 但允許用 ML / DL :

用 sklearn 的 SVM + HOG 特徵 訓練分類器 (資料夾中有現成的 hog_svm_rps.pkl)

用 TensorFlow Lite 跑訓練好的 MobileNetV2 (資料夾中有現成的 rps_mobilenetv2.tflite)

比較正確率：你會發現從 70% 跳到 90%+ — 但代價是模型大小與推論成本

正是這個對比讓你理解：什麼問題該用古典、什麼問題該換 AI。